

Software Test Plan

Project: AssetFlow

1. Test Objective

This test plan ensures all implemented functional requirements of the AssetFlow application remain working correctly after refactoring from layer-based to vertical slice architecture. Coverage includes authentication, asset management, borrow/transaction system, and role-based access control across backend (Spring Boot), web (React), and mobile (Kotlin) tiers.

2. Functional Requirements Coverage

FR1: User Registration

- **Requirement:** New users can register with email and password
- **Expected Behavior:** Email must be unique, password hashed, role assigned, JWT returned
- **Test Coverage:** Unit + Integration

FR2: User Login

- **Requirement:** Users authenticate with email/password, receive JWT token
- **Expected Behavior:** Correct credentials→200+JWT, incorrect→401, valid token persists
- **Test Coverage:** Unit + Integration + UI

FR3: Role-Based Access Control (RBAC)

- **Requirement:** Officers/Admins see full features; Students see limited features
- **Expected Behavior:** Admin routes blocked for students (403), student app UI restricted
- **Test Coverage:** Integration + Functional

FR4: View Assets

- **Requirement:** List all available assets with filters (status, category, search)
- **Expected Behavior:** Pagination, filtering, real-time updates
- **Test Coverage:** Unit + Integration + UI

FR5: Asset Details

- **Requirement:** View full asset information, related transactions
- **Expected Behavior:** Display serial number, category, status, borrow history
- **Test Coverage:** Integration + UI

FR6: Submit Borrow Request

- **Requirement:** Students submit borrow request with due date
- **Expected Behavior:** Request created (status=PENDING), notification sent to admin
- **Test Coverage:** Unit + Integration + UI

FR7: Approve/Reject Borrow Request

- **Requirement:** Admin approves or rejects pending requests
- **Expected Behavior:** Status transitions, user notified via email
- **Test Coverage:** Unit + Integration

FR8: Return Asset

- **Requirement:** Mark borrowed asset as returned
- **Expected Behavior:** Transaction status→RETURNED, return_date set, admin notified
- **Test Coverage:** Unit + Integration

FR9: View Transaction History

- **Requirement:** Users see their borrow requests; admins see all
- **Expected Behavior:** Pagination, status filtering, date range search
- **Test Coverage:** Unit + Integration

FR10: Logout

- **Requirement:** Users can logout and clear session/token

- **Expected Behavior:** Token removed client-side, subsequent requests rejected
- **Test Coverage:** UI + Functional

3. Test Cases

Slice 1: Authentication & Security

TC ID	Test Case Name	Input	Steps	Expected Output	Priority
TC1.1	Register new user	email=" student@university.edu ", password="SecureP@ss123"	1. Submit registration form 2. Verify DB for new user	HTTP 201, JWT in response, password hashed, role=USER	HIGH
TC1.2	Register duplicate email	email=" student@university.edu " (exists)	1. Try registration with existing email	HTTP 400, "Email already registered"	HIGH
TC1.3	Invalid password format	password="weak"	1. Submit weak password	HTTP 400, validation error	MEDIUM
TC1.4	Login with correct credentials	email=" student@university.edu ", password="SecureP@ss123"	1. POST /api/auth/login	HTTP 200, JWT token in response	HIGH
TC1.5	Login with wrong password	email=" student@university.edu ", password="Wrong"	1. POST /api/auth/login	HTTP 401, "Invalid credentials"	HIGH
TC1.6	Login non-existent user	email=" notexist@test.com "	1. POST /api/auth/login	HTTP 401, error message	HIGH
TC1.7	Get current user (/me)	Valid JWT token	1. GET /api/auth/me with JWT in Authorization header	HTTP 200, user object (id, email, name, role)	HIGH
TC1.8	Get /me without token	No JWT	1. GET /api/auth/me without	HTTP 401	HIGH

TC ID	Test Case Name	Input	Steps	Expected Output	Priority
			Authorization header		
TC1.9	Get /me with expired token	Expired JWT	1. GET /api/auth/me with old JWT	HTTP 401	MEDIUM
TC1.10	Logout	Valid JWT	1. POST /api/auth/logout	HTTP 200, "Logout successful"	MEDIUM

Slice 2: Asset Management

TC ID	Test Case Name	Input	Steps	Expected Output	Priority
TC2.1	List all assets	None	1. GET /api/assets	HTTP 200, array of asset objects	HIGH
TC2.2	Filter assets by status	status=AVAILABLE	1. GET /api/assets?status=AVAILABLE	HTTP 200, only AVAILABLE assets	HIGH
TC2.3	Filter assets by category	category_id=1	1. GET /api/assets?category=1	HTTP 200, filtered by category	MEDIUM
TC2.4	Get asset detail	assetId=5	1. GET /api/assets/5	HTTP 200, full asset object with relations	HIGH
TC2.5	Asset not found	assetId=999	1. GET /api/assets/999	HTTP 404, "Asset not found"	MEDIUM
TC2.6	Pagination	page=1&limit=10	1. GET /api/assets?page=1&limit=10	HTTP 200, 10 items maximum	LOW

Slice 3: Borrow/Transaction System

TC ID	Test Case Name	Input	Steps	Expected Output	Priority
TC3.1	Submit borrow request	assetId=1, dueDate="2026-06-01"	1. POST /api/assets/1/borrow with userId (from JWT)	HTTP 201, Transaction created, status=PENDING	HIGH
TC3.2	Borrow already-borrowed asset	assetId (status=BORROWED)	1. Try to borrow unavailable asset	HTTP 400, "Asset not available"	HIGH
TC3.3	Submit with past due_date	dueDate="2026-04-01" (past)	1. Submit request with past date	HTTP 400, validation error	MEDIUM
TC3.4	Approve borrow request (admin)	requestId=1, status="APPROVED"	1. Admin: PATCH /api/assets/borrow-requests/1 with APPROVED	HTTP 200, status→APPROVED, asset.status→BORROWED	HIGH
TC3.5	Non-admin approve attempt	requestId=1, student JWT	1. Student: PATCH /api/assets/borrow-requests/1	HTTP 403, "Forbidden"	HIGH
TC3.6	Reject borrow request (admin)	requestId=1, status="REJECTED"	1. Admin: PATCH /api/assets/borrow-requests/1 with REJECTED	HTTP 200, status→REJECTED, asset still AVAILABLE	MEDIUM
TC3.7	Return asset (admin)	requestId=1, status="RETURNED"	1. Admin: PATCH /api/assets/borrow-requests/1 with RETURNED	HTTP 200, status→RETURNED, asset.status→AVAILABLE, return_date set	HIGH
TC3.8	View my requests (student)	userId=1	1. Student: GET /api/assets/my-requests	HTTP 200, only this student's requests	HIGH
TC3.9	View all requests (admin)	None	1. Admin: GET /api/assets/borrow-requests	HTTP 200, all requests	HIGH
TC3.10	Approve request→Email sent	requestId=1	1. Submit + Approve borrow request 2. Check email	Email receipt with borrow details	MEDIUM

Slice 4: Authorization & Security

TC ID	Test Case Name	Input	Steps	Expected Output	Priority
TC4.1	Admin access admin endpoint	admin JWT to /api/admin	1. Admin: GET /api/admin/*	HTTP 200	HIGH
TC4.2	Student access admin endpoint	student JWT to /api/admin	1. Student: GET /api/admin/*	HTTP 403	HIGH
TC4.3	Protected endpoint no token	No JWT to protected route	1. GET /api/assets without token	HTTP 401	HIGH
TC4.4	Cross-user request	User1 JWT, request User2's data	1. User1 tries to access User2's transactions	HTTP 403 or filtered by current user	MEDIUM

4. Test Execution Plan

Backend (JUnit 5 + Mockito + TestContainers)

Duration: 2-3 hours

Environment: Maven, PostgreSQL container, H2 in-memory for fast tests

Unit Tests (1.5 hours):

- AuthServiceTest: Register, login, JWT generation, password hashing
- BorrowServiceTest: Status transitions, validation
- AssetServiceTest: Filtering logic
- AuthenticationUtilTest: Current user extraction

Integration Tests (1.5 hours):

- AuthControllerIT: Login, register, /me endpoints (TestContainers PostgreSQL)
- AssetControllerIT: List, filter, detail endpoints
- BorrowControllerIT: Submit, approve, return borrow requests
- SecurityIT: Role-based access, JWT validation

Execution:

```
cd backend  
./mvnw clean test
```

Web Frontend (Vitest + React Testing Library)

Duration: 1-2 hours (time-permitting)

Unit Tests:

- AuthContext test: Login, logout, role state
- AssetCard test: Rendering, props
- BorrowRequestCard test: Status display

Integration Tests:

- LoginPage test: Form submission, navigation
- DashboardPage test: Load assets, filter, admin check
- BorrowRequestsPage test: Load requests, approve/reject actions

Execution:

```
cd web  
bun run test
```

Mobile (JUnit 5 + Espresso)

Duration: 1.5 hours (time-permitting)

Unit Tests:

- TokenManagerTest: Save, retrieve, clear JWT

UI Tests:

- LoginActivityTest: Text entry, button click, navigation
- DashboardActivityTest: List rendering, permission checks

Execution:

```
cd mobile
./gradlew test
./gradlew connectedAndroidTest
```

5. Manual Regression Testing Plan

Duration: 2-3 hours

Tester: QA team

Environment: Local backend (8080), web dev (5173), mobile emulator (8081)

Test Script Execution Flow

Session 1: Authentication (30 minutes) ✓ ALL PASSED

1. Register new student ✓ PASSED

- Email: newstudent@test.edu, Password: TestPass123
- Verify: Account created, can login, JWT stored
- Result: Account successfully created and login verified

2. Register duplicate email ✓ PASSED

- Try register with existing email
- Verify: Error message shown
- Result: Error message displayed correctly

3. Login as student ✓ PASSED

- Email: student@test.edu, Password: (correct)
- Verify: Dashboard loads, no admin options visible
- Result: Dashboard loaded with student-only interface

4. Login as admin ✓ PASSED

- Email: admin@university.edu, Password: (correct)
- Verify: Dashboard loads WITH admin panel visible
- Result: Admin dashboard loaded with admin controls visible

5. Logout ✓ PASSED

- Click logout
- Verify: Redirected to login, token cleared from localStorage
- Result: Successfully logged out and token cleared

Session 2: Asset Management (30 minutes) ALL PASSED

1. View all assets PASSED

- As student: GET /api/assets
- Verify: List displays 10+ assets with cards
- Result: Asset list displayed with 10+ items

2. Filter by status PASSED

- Select "Available" filter
- Verify: Only AVAILABLE assets show
- Result: Filter applied successfully, showing only AVAILABLE assets

3. Filter by category PASSED

- Select "Laboratory Equipment"
- Verify: Only items in category show
- Result: Category filter applied, showing filtered results

4. View asset detail PASSED

- Click on "Microscope" asset
- Verify: Full details, serial number, category shown
- Result: Asset detail page displayed with all information

Session 3: Borrow System (45 minutes) ALL PASSED

1. Submit borrow request PASSED

- As student: Click borrow on "Beaker Set"
- Set due date: 2026-06-01
- Verify: Request submitted, appears in "MyTransactions" as PENDING
- Result: Borrow request submitted and visible with PENDING status

2. Admin approves request PASSED

- As admin: Go to Borrow Requests
- Click Approve on student's request
- Verify: Status→APPROVED, beaker set→BORROWED
- Result: Request approved, asset status changed to BORROWED

3. Verify student sees approval PASSED

- As student: Refresh MyTransactions
- Verify: Request shows APPROVED status
- Result: Student observed updated APPROVED status

4. Admin returns asset PASSED

- As admin: Click Return on approved request

- Verify: Status→RETURNED, beaker set→AVAILABLE
- Result: Asset returned successfully, status changed to AVAILABLE

5. Reject borrow request ✓ PASSED

- As admin: Submit new borrow, then Reject
- Verify: Request shows REJECTED, asset still AVAILABLE
- Result: Request rejected successfully, asset remained AVAILABLE

Session 4: Authorization (15 minutes) ✓ **ALL PASSED**

1. Student cannot access admin endpoints ✓ PASSED

- As student: Try to access /api/admin/stats
- Verify: HTTP 403 or page not found
- Result: Access denied correctly, HTTP 403 returned

2. Student UI has no admin buttons ✓ PASSED

- As student: View dashboard
- Verify: No "Manage Requests" or admin controls visible
- Result: Student interface verified, no admin controls present

3. Admin sees all requests ✓ PASSED

- As admin: View Borrow Requests
- Verify: Shows requests from all students, not just their own
- Result: Admin view showing all requests from all students